

QUINTAURIS

HACKATHON

NEORV32 on Arty A7-100T
Session final



Agenda



Part 1 — Arty A7-100T (2h 30min)

- **Exercise 1 — Knight Rider:** light effect on the Arty A7-100T LEDs, controlled with the buttons
- **Exercise 2 — Choosing the Right Processor:** adjust the NEORV32 hardware configuration so a given program runs correctly

Part 2 — QNTAF (1h 30min)

- **Exercise 1 — Compiler Flag Optimization:** improve the benchmarks by tuning compiler and linker flags
- **Exercise 2 — HW/SW Architecture Selection:** choose the core + OS/environment for a connected smart plug
- **Exercise 3 — Feedback on QNTAF:** tool evaluation and improvement proposal



Part 1 — Arty A7-100T

Duration: 2h 30min

Exercise 1 — Knight Rider



The Challenge

Program a “Knight Rider”-style light effect on the 4 LEDs (LD4-LD7) of the Arty A7-100T, controlled with the board’s push-buttons.

1. Make a lit LED sweep from LD4 → LD7.
2. Turn it into a real bounce:
0,1,2,3,2,1,0,1,2,3...
3. Use the buttons to control it in real time:
one speeds up, one slows down, one reverses direction.

Details

Files to modify

exercises/ejercicio1/sw/main.c (C only — the hardware is already solved).

Success criteria

The light pattern moves as requested and responds to the three buttons.

Exercise 2 — Choosing the Right Processor



The Challenge

You are given an already compiled program (*sw/neorv32_exe.bin*) and its source code (*sw/main.c*). When you run it on the board, it does not work correctly.

1. Read *main.c* and figure out what the program needs from the processor to run correctly.
2. Open *neorv32_arty_top.vhd* and locate the *generic map* of *neorv32_top*.
3. Add the missing generic (look up its exact name in *rtl/core/neorv32_top.vhd*).
4. Generate a new bitstream, program it, and upload the same *.bin* (no need to compile anything) to check the result.

Details

Files to modify

exercises/ejercicio2/src/neorv32_arty_top.vhd (VHDL only — the software is already compiled, do not touch it).

Success criteria

When you run the program, the console shows a sequence of correctly computed numbers, instead of staying silent or showing an illegal instruction error.



Part 2 — QNTAF

Duration: 1h 30min

Exercise 1 — Compiler Flags



Objective

For each benchmark (CoreMark, Dhrystone, Embench IoT size and speed), on the core combinations that support it (Nuclei N900 and/or Generic Core), try to improve on the default compilation profile (Default GCC / Default LLVM) by modifying the compiler and linker flags.

You can also explore switching toolchain (GCC vs. LLVM) where available.

Task

1. Run each benchmark with its default profile and record the result as the baseline.
2. Create and tune a new compilation profile until you improve the result, without breaking correct execution.
3. Document, per benchmark and core: baseline vs. optimized profile, flags used, and the (%) improvement obtained.

Deliverable

Benchmark × core × profile table with flags and results, plus a technical justification for each improvement.

Exercise 2 — HW/SW Architecture Selection



Role and Use Case

Role: a hardware architect at a consumer electronics company, deciding the combination of RISC-V core + OS/environment for the company's next product (Nuclei N900 on bare metal; Generic Core on bare metal, ThreadX, or Zephyr).

Use case: a Wi-Fi smart plug, manufactured at high volume. It must connect to the cloud and report electricity consumption. Per-unit cost and flash size are critical to the margin; also consider how much development time is saved by already having a networking stack available.

The use case must be defined only by its technical and business characteristics: do not use the Evaluate labels (real time, embedded, application) to describe or justify it.

Task

1. Choose the benchmarks relevant to the use case.
2. Evaluate the candidate core + OS combinations in QNTAF and compare the results.
3. Write a justified conclusion (recommended core + OS), backed by the data obtained.

Deliverable

Use case, methodology, a screenshot of the report (View), and a reasoned conclusion.

Exercise 3 — Feedback on the Tool



Strengths / Improvements

- 1. Strengths:** which features you found most useful or best designed, and why.
- 2. Areas for improvement:** what you found confusing, limited, or uncomfortable (usability, clarity, wait times, documentation).

Gaps and Proposal

- 3. Missing features:** what you missed (comparing results, exporting information, exploring core/OS/benchmark combinations).
- 4. Improvement proposal:** pick one of the points above and detail how you would implement it (which screen/flow, what information to add or remove).

·QUINTAURIS

quintauris.com